INSTITUTO VIRGINIO GÓMEZ
Construcción de Software

Sistema de ventas Aceros We-Corp

Hito 2: Diseño — diagramas y contratos OpenAPI

Integrantes: Bryan Vergara Santander
Aníbal Rodríguez
Docente: Enso Guidotti
Asignatura: Construcción de Software
Evaluación: Evaluación 3 (grupal)
Fecha: 9 de noviembre de 2026

Los Ángeles, Chile — 2026

Índice

1. Introducción.....3
2. Diagrama de clases de dominio.....4
3. Modelo entidad-relación.....5
3.1 Base de datos de Catálogo.....5
3.2 Base de datos de Ventas.....6
3.3 Justificación de la normalización.....7
4. Arquitectura.....8
5. Diagramas de secuencia.....9
5.1 Inicio de sesión y obtención del JWT.....9
5.2 Emitir nota de venta.....10
5.3 Flujos de error.....11
6. Diagramas complementarios.....12
6.1 Estados de la nota de venta.....12
6.2 Despliegue.....13
7. Contratos OpenAPI.....14
7.1 Diseño primero.....14
7.2 API de Catálogo e Identidad.....14
7.3 API de Ventas.....14
7.4 Modelo de errores.....15
7.5 Seguridad y versionamiento.....15
7.6 Registro de cambios.....15
8. Decisiones de arquitectura.....16
9. Trazabilidad inicial.....16
Anexo. Archivos fuente.....17

1. Introducción

Este documento corresponde al Hito 2 (Diseño) del sistema de ventas de Aceros We-Corp. Modela la solución definida en el Hito 1: dos APIs independientes en ASP.NET Core, cada una con su propia base de datos en SQL Server, y un cliente web en React que las consume.

- **API de Catálogo e Identidad:** autentica a los usuarios, emite el JWT y administra familias, productos, stock y usuarios.
- **API de Ventas:** valida el JWT y administra clientes y notas de venta; consume a la API de Catálogo para obtener precios y descontar o reponer stock.

Todos los diagramas se elaboraron como código en PlantUML y se versionan en el repositorio, en docs/diagramas, junto con su exportación en PNG y SVG. Los contratos OpenAPI están en docs/openapi y se escribieron antes de implementar las APIs. Las referencias RF, RNF, HU y CU corresponden a la ERS del Hito 1.

2. Diagrama de clases de dominio

El diagrama muestra las entidades del negocio agrupadas según la API a la que pertenecen, con sus atributos, métodos relevantes, relaciones y multiplicidades.

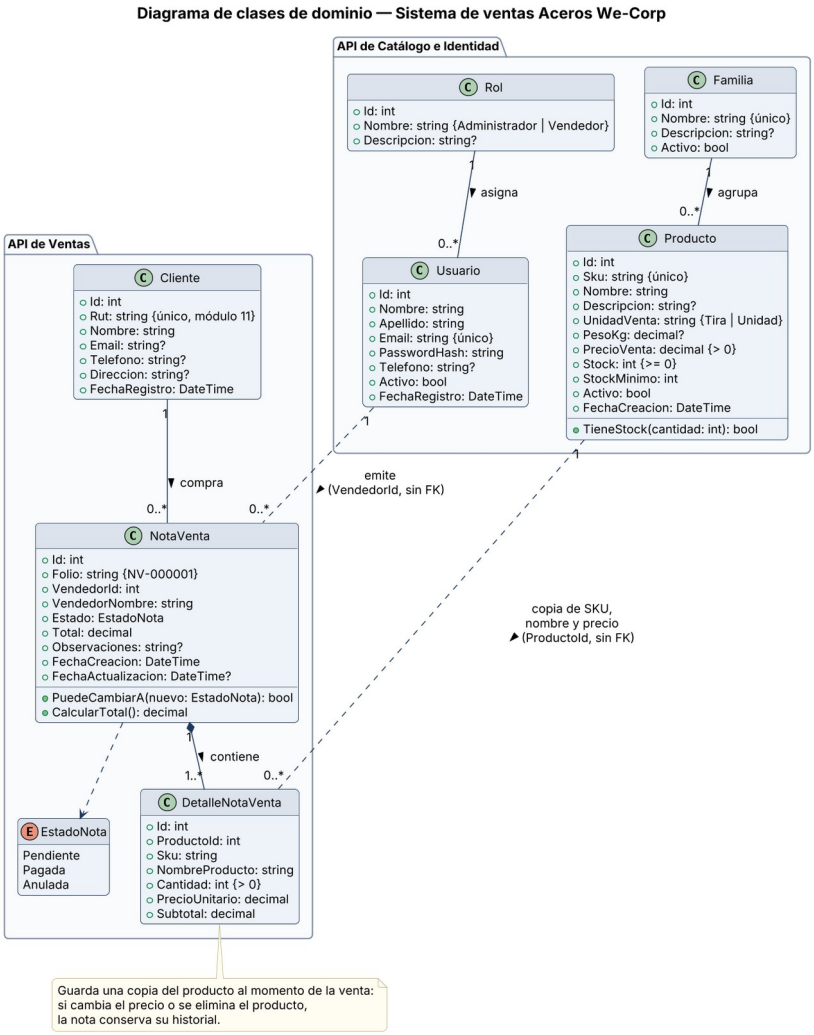


Figura 1. *Diagrama de clases de dominio.*

- **Catálogo:** un Rol agrupa a muchos Usuarios y una Familia agrupa a muchos Productos. Producto expone TieneStock(cantidad), que usa la regla de descuento de stock.
- **Ventas:** un Cliente tiene muchas Notas de venta y cada NotaVenta se compone de uno o más DetalleNotaVenta (composición: el detalle no existe sin su nota). NotaVenta expone PuedeCambiarA(estado), que solo permite pasar de Pendiente a Pagada o Anulada, y CalcularTotal().
- **Relaciones entre APIs:** NotaVenta referencia al vendedor (Usuario) y DetalleNotaVenta al Producto solo por su Id, porque viven en otra base de datos. Por eso se dibujan con línea punteada y el detalle guarda una copia del SKU, nombre y precio al momento de la venta.

3. Modelo entidad-relación

Cada API tiene su propia base de datos y solo accede a ella. Las tablas se crean con migraciones de Entity Framework Core. Las restricciones CHECK y los índices únicos protegen las reglas del negocio también a nivel de base de datos.

3.1 Base de datos de Catálogo

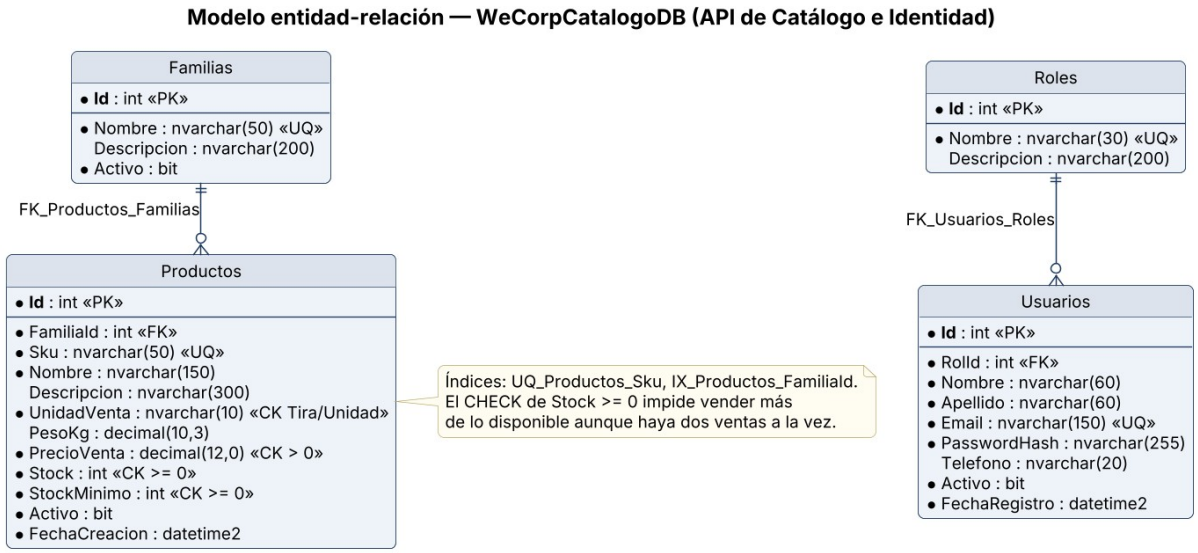


Figura 2. *Modelo entidad-relación de WeCorpCatalogoDB.*

Relación	Cardinalidad	Regla al eliminar
Roles → Usuarios	1 : N (un usuario tiene exactamente un rol)	Restrict: no se elimina un rol con usuarios
Familias → Productos	1 : N (un producto pertenece a una familia)	Restrict: no se elimina una familia con productos (409)

Tabla 1. *Relaciones de WeCorpCatalogoDB.*

3.2 Base de datos de Ventas

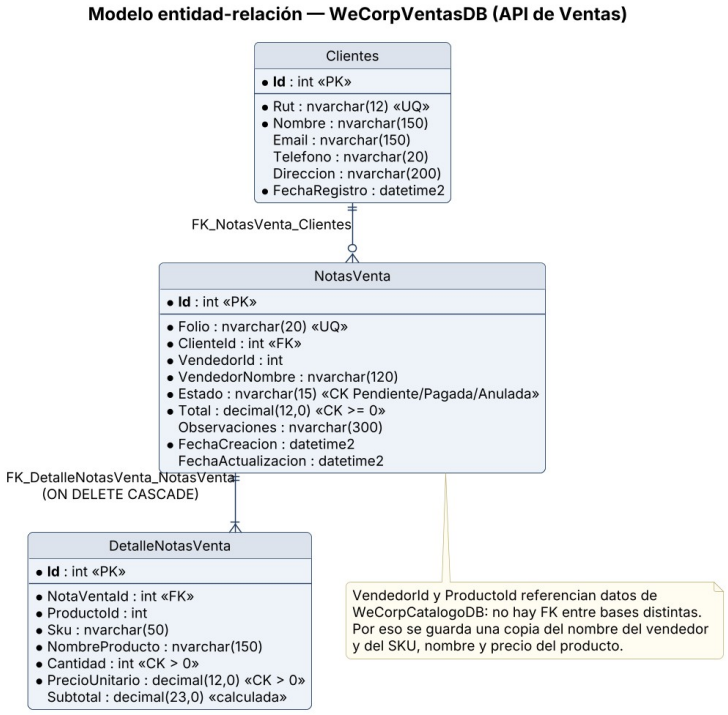


Figura 3. *Modelo entidad-relación de WeCorpVentasDB.*

Relación	Cardinalidad	Regla al eliminar
Clientes → NotasVenta	1 : N (una nota pertenece a un cliente)	Restrict: no se elimina un cliente con notas (409)
NotasVenta → DetalleNotasVenta	1 : 1..N (una nota tiene al menos una línea)	Cascade: el detalle se elimina con su nota

Tabla 2. *Relaciones de WeCorpVentasDB.*

3.3 Justificación de la normalización

Ambos modelos están en tercera forma normal (3FN):

- **1FN:** todos los atributos son atómicos; las líneas de una nota no se guardan como lista dentro de NotasVenta, sino como filas de DetalleNotasVenta.
- **2FN:** todas las tablas tienen clave primaria simple (Id), por lo que no hay dependencias parciales.
- **3FN:** no hay dependencias transitivas: el nombre de la familia está solo en Familias y el del rol solo en Roles; Productos y Usuarios guardan únicamente la clave foránea.

Se aceptan tres redundancias controladas, cada una con su motivo:

Dato	Motivo
Sku, NombreProducto y PrecioUnitario en DetalleNotasVenta	El producto vive en otra base y su precio cambia: la nota debe conservar los valores del momento de la venta (ADR 0001).
VendedorNombre en NotasVenta	El usuario vive en la base de Catálogo; se guarda el nombre para listar notas sin consultar otra API.
Total en NotasVenta y Subtotal calculada	Subtotal es una columna calculada por SQL Server (Cantidad × PrecioUnitario); Total se guarda para listar y filtrar notas sin sumar el detalle cada vez.

Tabla 3. *Redundancias controladas y su justificación.*

4. Arquitectura

Vista de contexto y contenedores: usuarios, cliente React, las dos APIs y sus bases de datos, con los protocolos de comunicación.

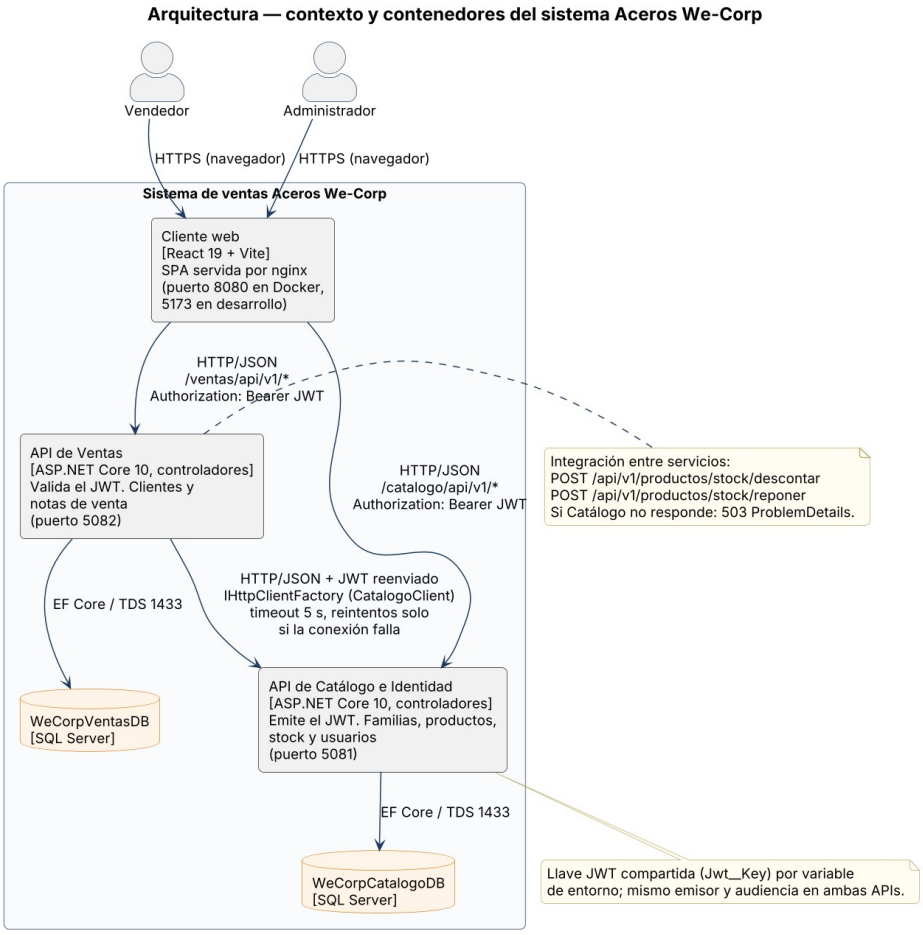


Figura 4. *Arquitectura del sistema (contexto y contenedores).*

Contenedor	Tecnología	Se comunica con	Protocolo
Cliente web	React 19 + Vite; nginx en Docker	Ambas APIs, mediante los prefijos /catalogo y /ventas	HTTP/JSON con Bearer JWT
API de Catálogo e Identidad	ASP.NET Core 10, EF Core 10	WeCorpCatalogoDB	TDS (SQL Server)
API de Ventas	ASP.NET Core 10, EF Core 10, IHttpClientFactory	WeCorpVentasDB y API de Catálogo	TDS; HTTP/JSON con el JWT reenviado
SQL Server	SQL Server Express o contenedor 2022	—	—

Tabla 4. *Contenedores del sistema.*

No hay sistemas externos: la integración entre servicios exigida por el enunciado es la de la API de Ventas con la API de Catálogo, mediante un cliente HTTP tipado con tiempo de espera de 5 segundos (ADR 0003).

5. Diagramas de secuencia

5.1 Inicio de sesión y obtención del JWT

El cliente envía las credenciales a la API de Catálogo, que verifica la contraseña con BCrypt y, si el usuario está activo, emite un JWT firmado con HMAC-SHA256 que dura 60 minutos (CU01, RF01).

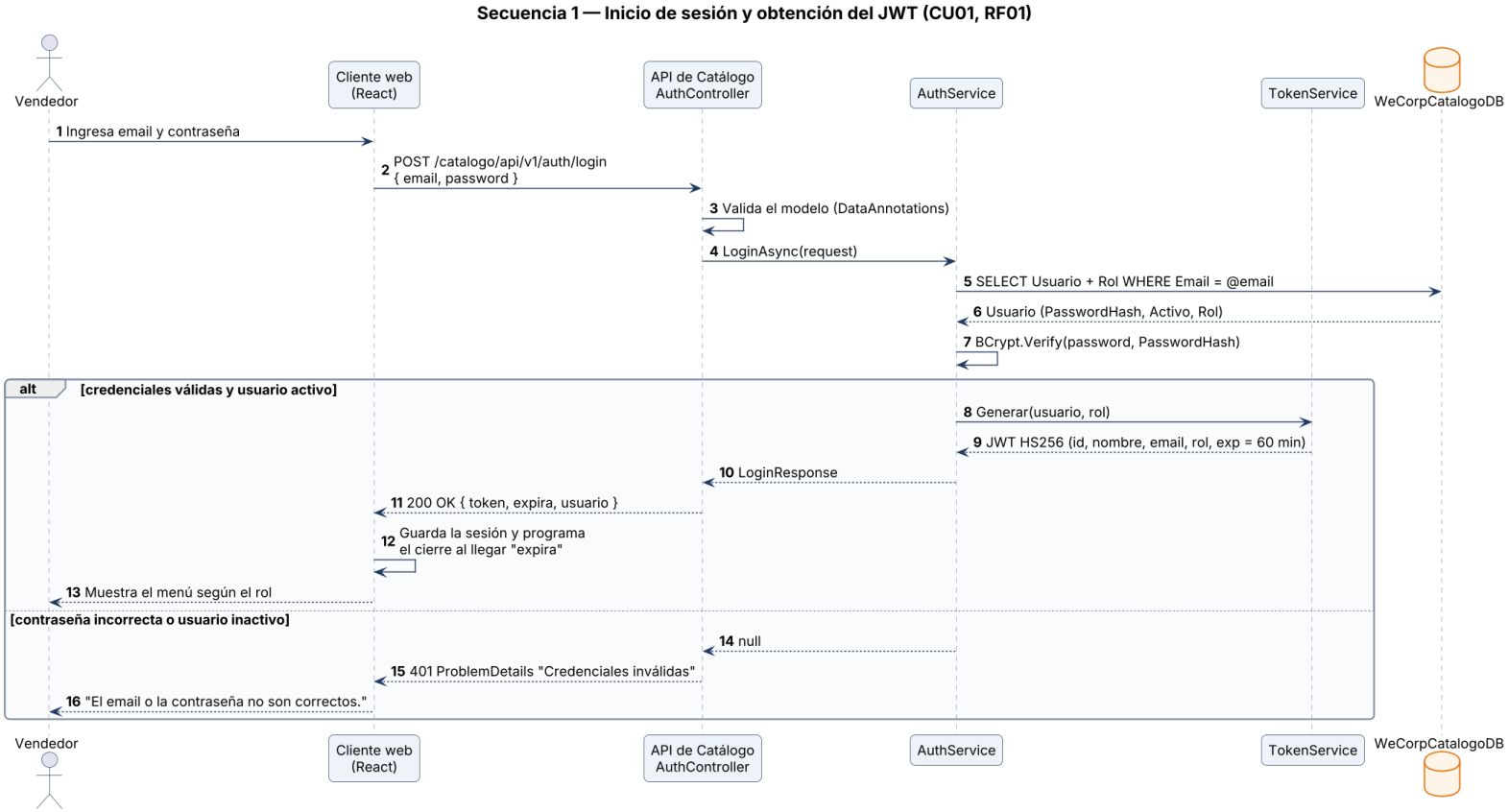


Figura 5. *Secuencia de inicio de sesión.*

5.2 Emitir nota de venta

Flujo de negocio completo: atraviesa el cliente, la API de Ventas, la API de Catálogo y ambas bases de datos. Catálogo verifica y descuenta el stock de todas las líneas en una sola operación y devuelve el precio vigente; recién entonces Ventas registra la nota (CU07, CU08; RF07, RF08).

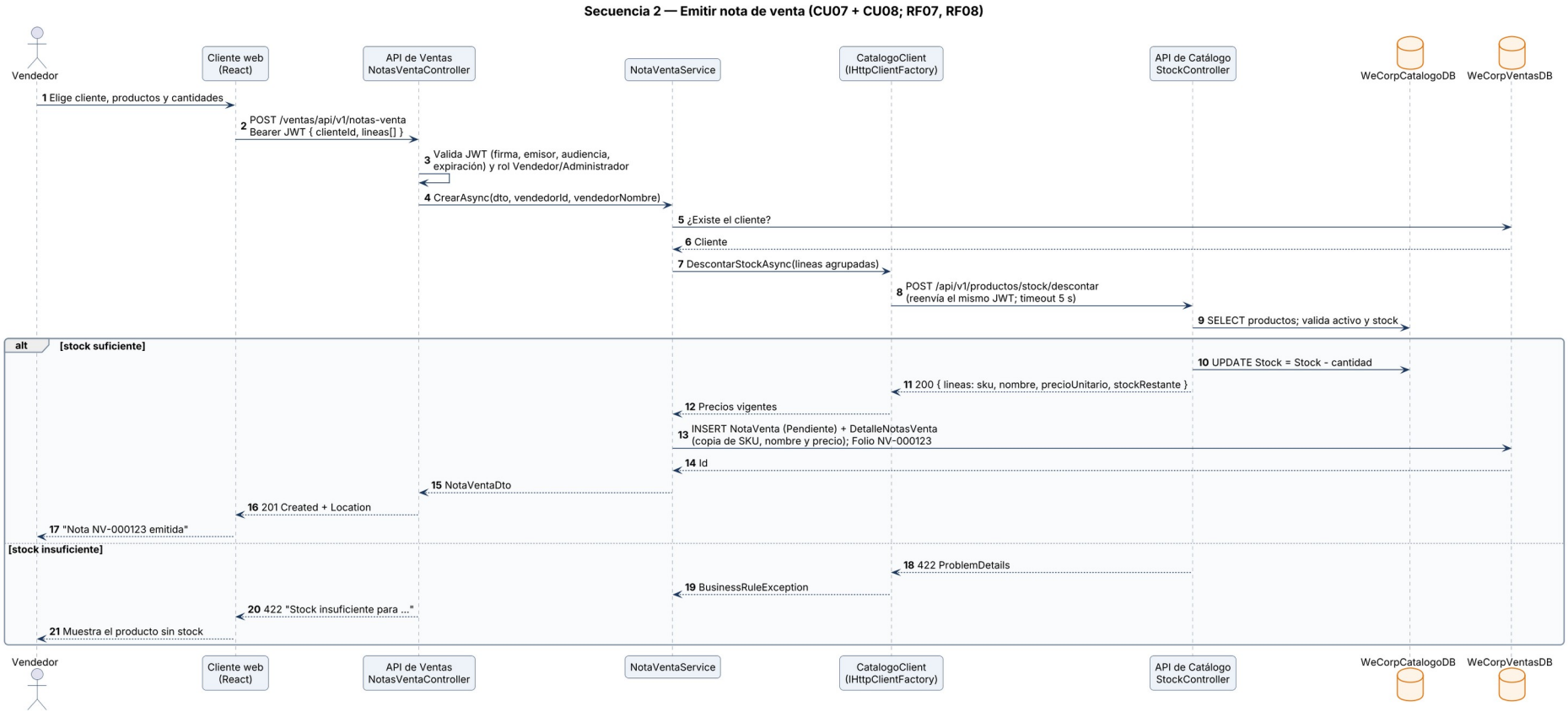


Figura 6. *Secuencia de emisión de una nota de venta.*

5.3 Flujos de error

Tres errores en un mismo diagrama: token vencido (401 y redirección al inicio de sesión), acceso denegado por rol (403) y API de Catálogo no disponible (503 sin registrar la nota).

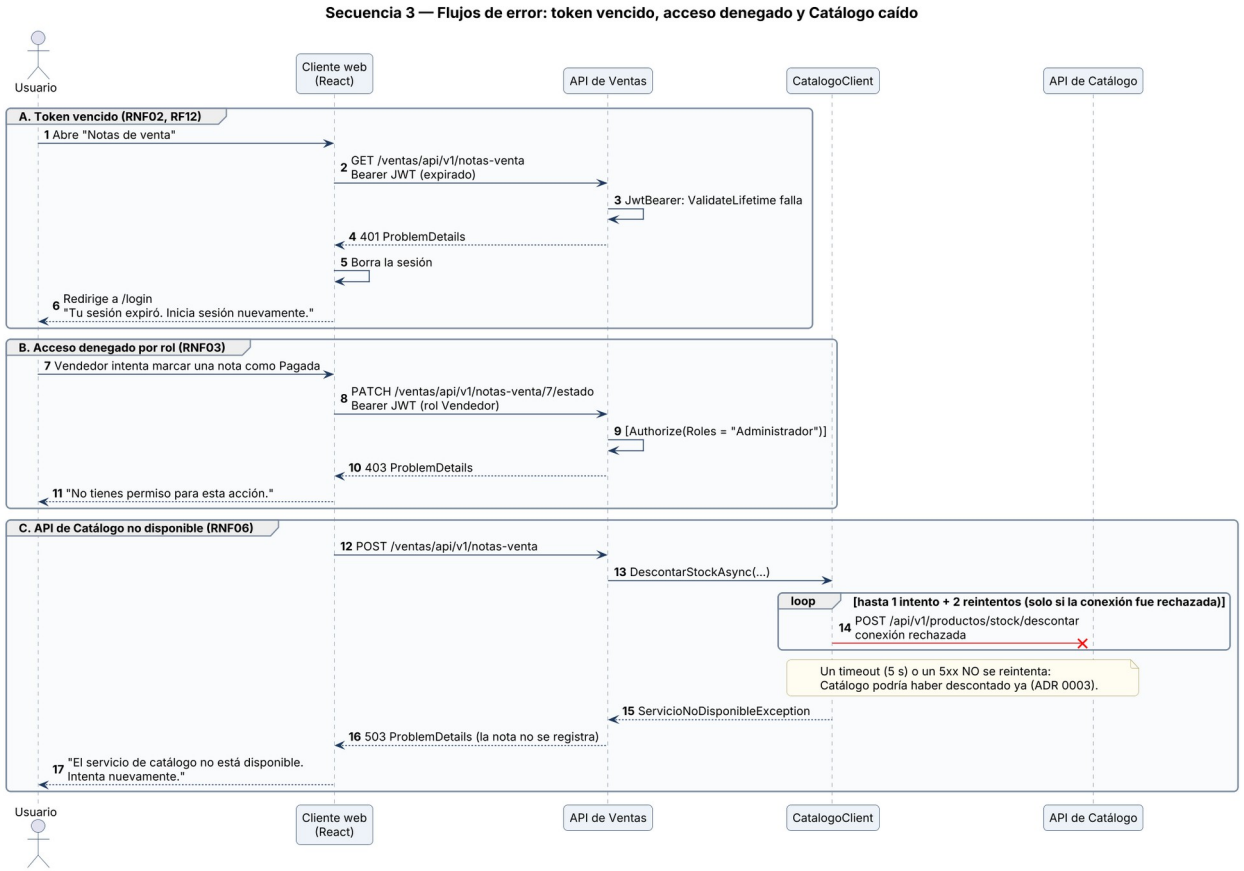


Figura 7. *Secuencia de los flujos de error.*

6. Diagramas complementarios

Aunque el Hito 2 no los exige, se adelantan los diagramas de estados y de despliegue, que forman parte de la entrega final.

6.1 Estados de la nota de venta

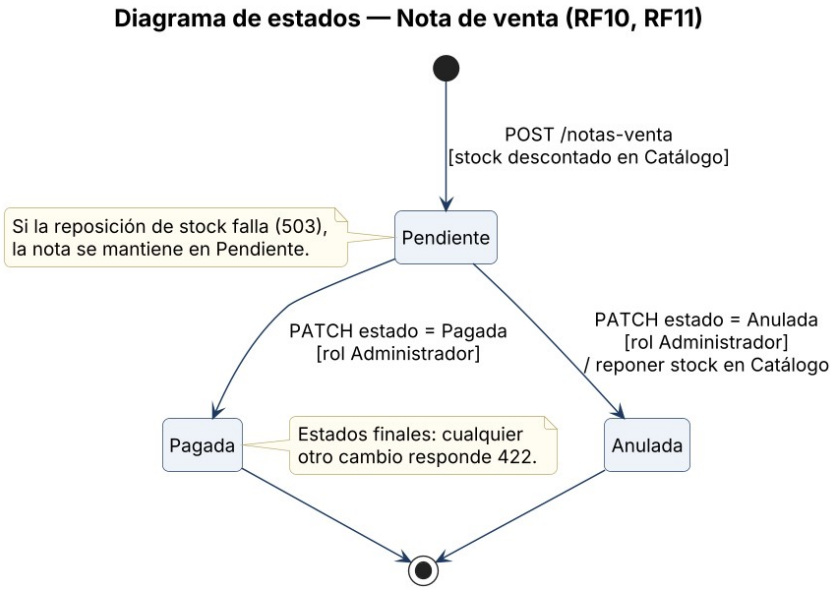


Figura 8. *Ciclo de vida de una nota de venta.*

Una nota nace Pendiente al descontarse el stock. Solo el Administrador puede pasarla a Pagada o Anulada; al anular se repone el stock. Pagada y Anulada son estados finales (RF10, RF11).

6.2 Despliegue

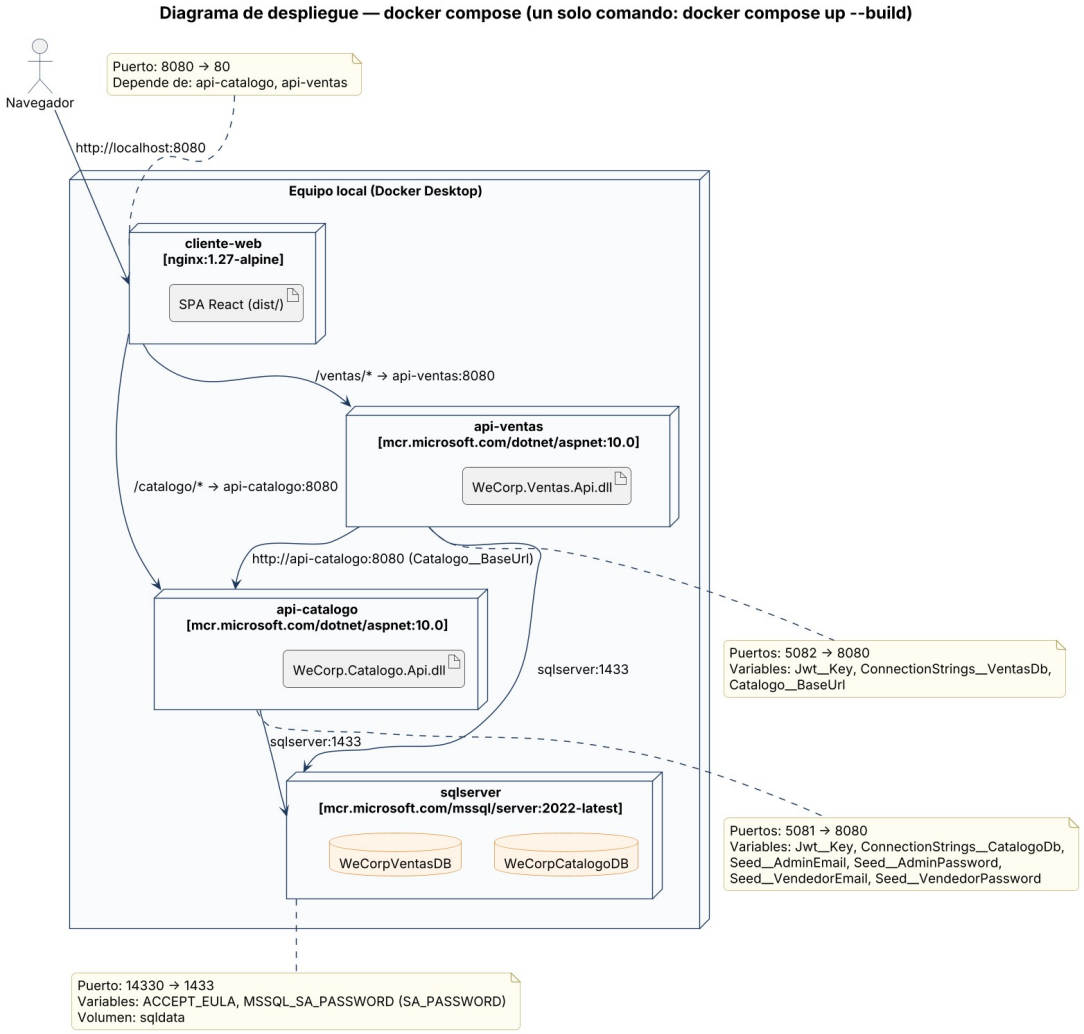


Figura 9. *Despliegue con docker compose.*

El sistema completo se levanta con docker compose up. Cada contenedor indica su puerto y las variables de entorno que necesita, sin valores secretos (RNF04, RNF07).

7. Contratos OpenAPI

7.1 Diseño primero

Los contratos se escribieron en OpenAPI 3.0.3 antes de programar y se validaron con Redocly CLI. Definen recursos, rutas, verbos, esquemas, códigos de estado, modelo de errores, seguridad y versión. Archivos: `docs/openapi/catalogo-v1.yaml` y `docs/openapi/ventas-v1.yaml`. La implementación debe coincidir con ellos y Swagger, en cada API, permite comprobarlo.

7.2 API de Catálogo e Identidad

Verbo	Ruta (/api/v1)	Rol	Éxito	Errores
POST	/auth/login	Público	200	400, 401
GET	/familias, /familias/{id}	Admin, Vendedor	200	401, 404
POST, PUT, DELETE	/familias[/{id}]	Admin	201, 204	400, 403, 404, 409
GET	/productos? buscar&familiaId&pagina&tamano	Admin, Vendedor	200	401
GET	/productos/{id}	Admin, Vendedor	200	401, 404
POST, PUT	/productos[/{id}]	Admin	201, 204	400, 403, 404, 409, 422
DELETE	/productos/{id}	Admin	204	403, 404
POST	/productos/stock/descontar	Admin, Vendedor	200	400, 404, 409, 422
POST	/productos/stock/reponer	Admin, Vendedor	200	400
GET, POST	/usuarios	Admin	200, 201	400, 403, 409, 422
GET, PUT	/usuarios/{id}	Admin	200, 204	400, 403, 404, 422
GET	/usuarios/roles	Admin	200	403
GET	/health (sin versión)	Público	200	503

Tabla 5. *Endpoints de la API de Catálogo e Identidad.*

7.3 API de Ventas

Verbo	Ruta (/api/v1)	Rol	Éxito	Errores
GET	/clientes?buscar&pagina&tamano	Admin, Vendedor	200	401
GET	/clientes/{id}	Admin, Vendedor	200	401, 404
POST, PUT	/clientes[/{id}]	Admin, Vendedor	201, 204	400, 404, 409
DELETE	/clientes/{id}	Admin	204	403, 404, 409

Verbo	Ruta (/api/v1)	Rol	Éxito	Errores
GET	/notas-venta? estado&desde&hasta&pagina&tamano	Admin, Vendedor*	200	400, 401
GET	/notas-venta/{id}	Admin, Vendedor*	200	401, 404
POST	/notas-venta	Admin, Vendedor	201	400, 404, 422, 503
PATCH	/notas-venta/{id}/estado	Admin	200	400, 403, 404, 422, 503
GET	/health (sin versión)	Público	200	503

Tabla 6. *Endpoints de la API de Ventas. (*) El Vendedor solo ve sus propias notas.*

7.4 Modelo de errores

Ambas APIs responden los errores en formato ProblemDetails (RFC 7807, application/problem+json) con title, status, detail e instance; los errores de validación agregan errors, con el problema de cada campo.

Código	Significado en el sistema
400	Datos inválidos (DataAnnotations; RUT con dígito verificador incorrecto)
401	Sin token, token alterado o vencido, o credenciales incorrectas
403	El rol del usuario no tiene permiso
404	El recurso no existe, o una nota de otro vendedor
409	Dato repetido (SKU, RUT, email, nombre), registro en uso o stock modificado por otra venta
422	Regla de negocio: stock insuficiente, producto inactivo o transición de estado no permitida
503	La API de Catálogo no respondió a tiempo o falló

Tabla 7. *Códigos de estado y su uso.*

7.5 Seguridad y versionamiento

- **Seguridad:** esquema Bearer (JWT) en ambos contratos. El token lo emite la API de Catálogo y ambas APIs validan firma, emisor, audiencia y expiración con la misma llave, entregada por la variable de entorno Jwt__Key (ADR 0002).
- **Versionamiento:** por segmento de URL (/api/v1), con Asp.Versioning. Un cambio incompatible se publicaría como /api/v2 sin romper a los clientes de la v1.

7.6 Registro de cambios

Los cambios a los contratos se registran en `docs/openapi/CHANGELOG.md`. Tras la revisión del diseño se agregaron:

- Ventas: GET /notas-venta responde 400 cuando el filtro estado no es Pendiente, Pagada ni Anulada.

- Catálogo: POST /productos/stock/descontar responde 409 si otra venta modificó el mismo stock al mismo tiempo, y POST /productos/stock/reponer omite los productos ya eliminados, para que una nota antigua se pueda anular.

8. Decisiones de arquitectura

Se registraron tres decisiones en `docs/adr/`:

ADR	Decisión	Principal consecuencia
0001	Dividir el sistema en dos APIs, cada una con su propia base de datos.	No hay FK entre bases: el detalle de la nota guarda una copia del producto.
0002	El JWT lo emite la API de Catálogo (HS256, 60 min) y ambas APIs lo validan con la misma llave; Ventas reenvía el token del usuario a Catálogo.	Una sola fuente de identidad; la llave debe protegerse como secreto.
0003	Integración Ventas → Catálogo con cliente HTTP tipado: timeout de 5 s, reintentos solo si la conexión falla, compensación si no se puede guardar la nota y concurrencia optimista sobre el stock.	Nunca se descuenta stock dos veces por un reintento; ante fallas, 503 sin registrar la nota.

Tabla 8. *Registros de decisiones de arquitectura.*

9. Trazabilidad inicial

Cada requerimiento funcional queda unido a su historia de usuario y a los endpoints que lo implementarán. La matriz completa, con la columna de pruebas, se mantiene en `docs/trazabilidad.md`.

RF	HU	Endpoint(s)	API
RF01	HU01	POST /auth/login	Catálogo
RF02	HU03	GET /productos, GET /productos/{id}	Catálogo
RF03	HU04	POST, PUT, DELETE /productos	Catálogo
RF04	HU05	GET, POST, PUT, DELETE /familias	Catálogo
RF05	HU06	GET, POST, PUT /usuarios; GET /usuarios/roles	Catálogo
RF06	HU07, HU08	GET, POST, PUT, DELETE /clientes	Ventas
RF07	HU09	POST /notas-venta	Ventas
RF08	HU09	POST /productos/stock/descontar (llamado por Ventas)	Ventas → Catálogo
RF09	HU10	GET /notas-venta, GET /notas-venta/{id}	Ventas
RF10	HU11	PATCH /notas-venta/{id}/estado	Ventas
RF11	HU11	POST /productos/stock/reponer (llamado por Ventas)	Ventas → Catálogo
RF12	HU02	Respuesta 401 de cualquier endpoint protegido	Ambas

Tabla 9. *Trazabilidad requerimiento → historia de usuario → endpoint.*

Anexo. Archivos fuente

Todos los archivos se entregan junto a este documento y se versionan en el repositorio:

Archivo	Contenido
docs/diagramas/clases-dominio.puml	Diagrama de clases de dominio
docs/diagramas/er-catalogo.puml, er-ventas.puml	Modelos entidad-relación
docs/diagramas/arquitectura.puml	Arquitectura (contexto y contenedores)
docs/diagramas/secuencia-login.puml	Secuencia de inicio de sesión
docs/diagramas/secuencia-emitir-nota.puml	Secuencia de emisión de nota de venta
docs/diagramas/secuencia-error-token.puml	Secuencia de flujos de error
docs/diagramas/estados-nota-venta.puml, despliegue.puml	Diagramas complementarios
docs/diagramas/*.png, *.svg	Exportaciones de cada diagrama
docs/openapi/catalogo-v1.yaml, ventas-v1.yaml	Contratos OpenAPI 3.0.3
docs/openapi/CHANGELOG.md	Registro de cambios de los contratos
docs/adr/0001…0003	Registros de decisiones de arquitectura

Tabla 10. *Archivos fuente del Hito 2.*

Los diagramas se pueden visualizar en plantuml.com/plantuml pegando el contenido del archivo .puml (cada archivo es independiente), o en VS Code con la extensión PlantUML. Los contratos se pueden abrir en editor.swagger.io.